

A Critic-in-the-Loop Multi-Agent Framework for Reducing Hallucinations in AI Research Reports

CSS5120 Computational Linguistics

CHEN Mingxin 225030242

ZHUANG Runsong 120090390

May 9, 2026

1. Introduction

1.1 Problem Statement

Large language model (LLM) systems can generate long-form research reports quickly, but they often produce factually weak or unverifiable claims (hallucinations), especially around numeric metrics like adoption rates, ROI, payback periods, and pricing. The problem we are solving is the lack of intermediate factual validation in autonomous research agents before their outputs are published.

1.2 Importance of the Problem

This issue is of high practical importance in business intelligence, strategy, and policy contexts, where a single unsupported number can severely mislead downstream decisions. As the length of automated reports increases, manual fact-checking becomes prohibitively expensive and inconsistent, negating the efficiency benefits of using AI agents.

1.3 Existing Work and Limitations

Current approaches to automated research include single-agent generation (which is fast but carries a high factual risk), RAG-based systems (which improve grounding, though "retrieved" does not equal

"verified"), and multi-agent writing pipelines. Existing frameworks rely on an open-loop pipeline (Plan -> Search -> Write -> Publish). The critical limitation in the current literature and codebase is a missing quality gate with explicit correction loops before publication. If the writing agent misinterprets data or hallucinates, the error is published immediately to the end-user.

1.4 Innovation and Contribution

Our primary contribution in this project is upgrading the original multi_agents LangGraph workflow in gpt-researcher by introducing a "Critic-in-the-loop" architecture. We transitioned the system from an open-loop pipeline to a self-correction closed-loop framework.

2. Methodology

2.1 Architectural Shift: Open-Loop to Closed-Loop

We utilized the LangGraph framework to transform the standard multi-agent research architecture from an open-loop pipeline into a self-correcting closed-loop system. Specifically, we injected a Critic agent directly between the writer and publisher nodes to serve as a final factual checkpoint. By implementing score-based routing, we replaced the traditional "write once, publish once" paradigm with

an automated revision loop that intercepts low-quality drafts before publication.

2.2 The Critic-in-the-Loop Workflow

The core mechanism of our methodology relies on a strict evaluation and routing protocol orchestrated by the Critic node:

Evaluation (Inputs & Outputs): After the writer generates the `full_draft`, the Critic reads the content and evaluates it based on predefined rules. It outputs three key artifacts into the shared state: a confidence score, a short risk report (fact-check summary), and optional targeted search queries for any missing evidence.

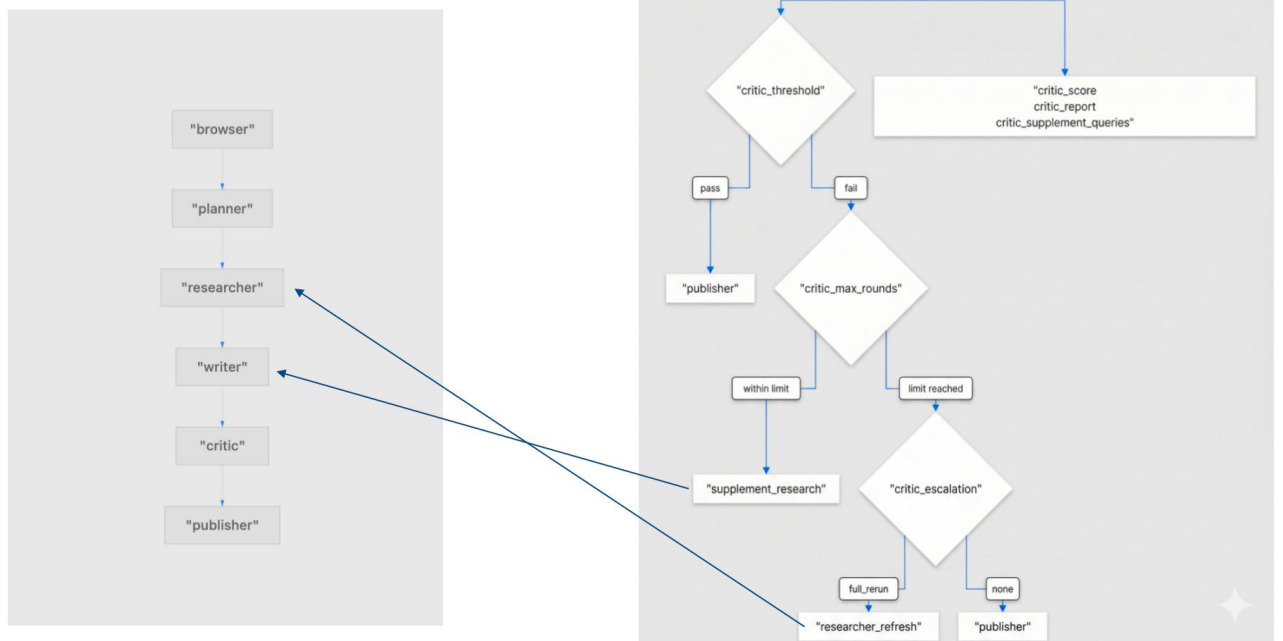
Conditional Routing: The LangGraph state machine uses these outputs to determine the next step dynamically:

Pass: If the score meets or exceeds the `critic_threshold`, the draft is approved and routed directly to the publisher.

Revise: If the score is below the threshold, the system checks the `critic_max_rounds` counter. If the loop budget permits, it routes to `supplement_research` (where the agent executes the newly generated search queries), loops back to the writer for revision, and returns to the critic for re-evaluation.

Escalate: If the system exhausts the maximum allowed rounds without achieving a passing score, the critic_escalation policy dictates the fallback behavior. It either triggers a full system rerun via the researcher_refresh node or force-publishes the best-effort draft with a warning flag.

System Architecture



2.3 Parameterized Control and State Memory

To technically support this dynamic loop, we extended the LangGraph memory architecture (multi_agents/memory/research.py) to track critical state variables such as iteration counters, historical scores, and generated queries. Furthermore, we exposed dynamic configurations via task.json to give users granular control over the

quality gate. Key parameters include `critic_threshold` (to define the passing grade) and `critic_max_rounds` (to cap the effort, prevent infinite loops, and control API compute costs).

```
multi_agents > {} task.json > ...
```

```
1  {
2    "query": "2025年中国AI应用市场: 中大型企业 (对象) 在客服与营销场景的落地率、单项目ROI与平均回本周期 (指标)",
3    "max_sections": 1,
4    "publish_formats": {
5      "markdown": true,
6      "pdf": false,
7      "docx": false
8    },
9    "include_human_feedback": false,
10   "follow_guidelines": false,
11   "model": "gpt-4o",
12   "guidelines": [
13     "The report MUST be written in APA format",
14     "Each sub section MUST include supporting sources using hyperlinks. If none exist, erase the sub section or rewrite",
15     "The report MUST be written in spanish"
16   ],
17   "verbose": false,
18   "critic_threshold": 82,
19   "critic_max_rounds": 1,
20   "critic_incremental_max_queries": 2,
21   "critic_escalation": "none",
22   "critic_append_to_report": true,
23   "enable_run_trace": true
```

Finally, to eliminate the "black-box" nature of autonomous agents, we implemented comprehensive `run_trace` artifacts (`run_trace.json` and `run_trace.md`). These logs record the exact execution path and state changes, ensuring the entire decision-making process remains transparent and observable.

2.4 Justification of Choices

We chose workflow engineering via LangGraph over static prompt engineering because a static prompt cannot magically synthesize facts missing from the initial search phase. Our multi-agent loop solves this by enabling the system to actively execute new, targeted web searches

(supplement_research) based on the Critic's specific feedback, providing a robust mechanism to dynamically retrieve missing information.

3. Results

3.1 Key Findings

The system successfully executes the critic loop and generates traceable artifacts, including run_trace.json and run_trace.md.

```
1  # Run Trace
7  ## Workflow Graph
8
9  ```mermaid
10 flowchart TD
11     browser --> planner --> human
12     human -->|accept| researcher
13     human -->|revise| planner
14     researcher --> writer --> critic
15     critic -->|pass| publisher
16     critic -->|supplement| supplement_research --> writer
17     critic -->|full_rerun| researcher_refresh --> writer
18     critic -->|publish_anyway| publisher
19
```

```
22  ## Executed Path
23
24  1. `2026-05-05T09:46:51Z` -> `browser`
25  2. `2026-05-05T09:46:56Z` -> `planner`
26  3. `2026-05-05T09:46:56Z` -> `human`
27  4. `2026-05-05T09:47:50Z` -> `researcher`
28  5. `2026-05-05T09:48:06Z` -> `writer`
29  6. `2026-05-05T09:53:20Z` -> `critic` (critic_score=78.0,
    supplement_queries=2)
30  7. `2026-05-05T09:55:41Z` -> `supplement_research` (increment_cycle=1)
31  8. `2026-05-05T09:56:11Z` -> `writer`
32  9. `2026-05-05T09:56:22Z` -> `critic` (critic_score=78.0,
    supplement_queries=2)
33  10. `2026-05-05T09:56:22Z` -> `publisher`
```

In a documented real-world execution, the total runtime was 933,621 ms (approximately 15.6 minutes) with a successful exit code 0. The initial evaluation by the critic node yielded a score of 78.0 and triggered 2 supplement queries. The workflow correctly routed to the supplement_research node and returned to the writer.

3.2 Meaning of the Results

Even after the supplemental loop, the score remained at 78.0. This occurred due to evidence quality bottlenecks (e.g., Content too short or empty, PDF read failures, timeouts) and the fact that high-risk numeric claims require highly accessible and strong evidence.

However, the system successfully generated a fact-check summary that flagged uncertainties regarding adoption percentages, ROI metrics, payback periods, and pricing. This transparent logging

ensures that end-users are explicitly warned about unsupported claims, achieving our goal of verifiable and safe research outputs.

4. Discussion

4.1 Potential Applications

This architecture is highly applicable to enterprise internal market intelligence reports and strategic analysis memos. It ensures that outputs are not only fast but also verifiable, traceable, and correction-oriented.

4.2 Limitations

There are several known limitations. First, the `critic_score` is currently LLM-evaluated, which relies on subjective assessment rather than fully evidence-weighted deterministic scoring. Second, external source quality is highly unstable; issues like PDF read failures or site timeouts severely limit the effectiveness of the supplement loop and increase runtime overhead. Third, the correction capabilities are constrained by the loop budget (e.g., `critic_max_rounds=1`).

4.3 Improvements and Next Steps

To address these limitations, our concrete next steps include improving source quality controls through domain allowlists, fast

fallbacks, and stricter content-length filters. Furthermore, we plan to evolve the critic output into claim-level structured verification with explicit evidence links. Finally, we will add a run-level metrics dashboard to track score deltas, supplement hit-rates, and runtime by stage.

5. References

Elovic, A. (2023). *GPT-Researcher: An autonomous agent designed for comprehensive online research*[Source Code]. GitHub.

LangChain AI. (2024). *LangGraph Documentation: Building Stateful Multi-Actor Applications*.

Richards, T. B. (2023). *Auto-GPT: An Autonomous GPT-4 Experiment* [Source Code]. GitHub.

Shao, Y., Jiang, Y., Kanell, T. A., Xu, P., Khattab, O., & Lam, M. S. (2024). *Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models* (Stanford STORM). arXiv preprint arXiv:2402.14207.